Этап 9. Полноценная обработка запросов.

Содержание

1. Структура этапа 2
2. Текущая реализация: 2
3. Добавление контекста для HTTP обработчиков 2
4. Реализация обработки запросов (пример): 4
   - 4. 1. Валидация запроса. 4
   - 4. 2. Получение подключения к базе данных 4
   - 4. 3. Бизнес логика. 5
   - 4. 4. Вернуть результат 6

Задачи 7

В этапах 3,4,6 мы разрабатывали взаимодействие приложения с базой данных. В этапе 8 мы разрабатывали обработку ошибок при создании сущностей. Теперь мы можем перейти к полноценной обработке запросов.

1. Структура этапа

Файл: `Этап 9. Полноценная обработка запросов` содержит только пример и основные концепции, как реализовать полноценную обработку запросов.

Для того, чтобы реализовать этот этап, вам необходимо будет пройти следующие шаги, по ходу решения задачи:

- Конвертация типов вашего приложения в sql и наоборот.
- Спецификация для гибких запросов.
- Envelope — обертка для возврата результата HTTP обработчика.

2. Текущая реализация:

Текущая реализация функции `add_item` не сохраняет созданный элемент в базе данных и не использует какую-либо валидацию:

```
pub async fn add_item(
    Json(payload): Json<AddItemPayload>,
    Context(context): Context<'_, RequestContext>,
) -> Response {
    println!(
        "Adding item: {} with price: {}",
        payload.name, payload.price
    );

    let item = Item::create_new(payload.name, payload.price);

    println!(
        "Item created with ID: {}, Name: {}, Price: {}, Created At: {}",
        item.id, item.name, item.price, item.created_at
    );

    let mut response = Response::new(Status::OK);
    response.set_text("added item");
    return response;
}
```

Необходимо доработать функциональность и сделать следующее:

1. Валидировать входные данные перед созданием элемента.
2. Сохранить созданный элемент в базе данных.

3. Добавление контекста для HTTP обработчиков

Чтобы иметь доступ к базе данных в HTTP обработчиках, необходимо добавить контекст в функцию обработчика.

Информация:

Что такое контекст?

Контекст — это дополнительная информация, которая передается в функцию обработчика. В контексте может хранится информация о каких-то настройках, ключах, подключениях к базе данных и т.д. Считайте, что контекст — это коробка с инструментами, которые могут быть использованы в функции обработчика.

Для нашего случая, нам может понадобиться контекст, содержащий пул подключений к базе данных. Это должна быть структура, которая будет хранить пул подключений к базе данных.

Её можно сделать, например, вот так:

```
#[doc = "HTTP контекст - коробка с инструментами."]
pub struct ItemsHttpContext {
    connection_pool: Arc<ManagedConnectionPool>,
}
```

Здесь мы создаем структуру `ItemsHttpContext`, которая будет содержать пул подключений к базе данных.

Можно так же добавить ей функции для получения и возврата подключения к базе данных, так будет удобнее.

```
impl ItemsHttpContext {
    pub fn new(connection_pool: Arc<ManagedConnectionPool>) -> Self {
        Self { connection_pool }
    }

    #[doc = "Получить подключение к базе данных."]
    async fn get_connection(
        &self,
    ) -> Result<Client, Box<dyn std::error::Error>> {
        let connection = self.connection_pool.acquire_connection().await?;
        Ok(connection)
    }

    #[doc = "Вернуть подключение к базе данных."]
    async fn take_back(&self, connection: Client) {
        self.connection_pool.return_connection(connection).await;
    }
}
```

Подключать контексты в HTTP обработчики в фреймворке `Ohkami` можно следующим образом:

```
use ohkami::fang::Context;
// ^^^^^^^^^^^^^^^^^^^^^^
// необходимый импорт для использования контекста в HTTP обработчиках

pub async fn add_item(
    Json(payload): Json<AddItemPayload>,
    Context(context): Context<'_, ItemsHttpContext>,
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
//  это подключение контекста в функцию обработчика.
) -> Response
```

4. Реализация обработки запросов (пример):

Теперь, когда HTTP обработчики получили доступ к контексту, у нас есть все необходимое, чтобы реализовать обработку запросов.

В основном любую обработку запросов можно разбить на несколько этапов:

1. **Валидация запроса.**
2. **Получение подключения к базе данных.**
3. **Бизнес логика.**
4. **Формирование ответа.**

4. 1. Валидация запроса.

Рекомендуется выполнить задание: «Envelope — обертка для возврата результата HTTP обработчика», перед выполнением этого шага.

Первым этапом HTTP обработки старайтесь приоритетно проверить корректность входных данных. Если данные некорректны — нам не нужно продолжать обработку запроса и выполнять дорогостоящие операции, например, обращаться к базе данных.

```
pub async fn add_item(
    Json(payload): Json<AddItemPayload>,
    Context(context): Context<'_, ItemsHttpContext>,
) -> Response {
    // Создаем объекты значения, процедура создания которых уже
    // проверяет входные данные
    let name_clone = payload.name.clone();
    let name: Name = match Name::new_with_format_and_length_check(
        payload.name,
        vec![FormatOptions::Trim, FormatOptions::FirstLetterToUpper],
        MIN_ITEM_NAME_LENGTH,
        MAX_ITEM_NAME_LENGTH,
    ) {
        Err(e) => return Envelope::<()>::bad_request(Some(e.to_string())),
        Ok(name) => name,
    };

    let price: Price = match Price::new(payload.price) {
        Err(e) => return Envelope::<()>::bad_request(Some(e.to_string())),
        Ok(price) => price,
    };
```

4. 2. Получение подключения к базе данных

- **Рекомендуется выполнить** задание: «Envelope — обертка для возврата результата HTTP обработчика», перед выполнением этого шага.
- **Рекомендуется выполнить** задание: «Спецификация для гибких запросов», перед выполнением этого шага.

- **Рекомендуется выполнить** задание: «Конвертация типов вашего приложения в sql и наоборот», перед выполнением этого шага.

После этапа проверки входных данных мы можем получить доступ к базе данных, чтобы сохранить новый товар или выполнить другую дополнительную операцию.

```
let connection = match context.get_connection().await {
    Ok(connection) => connection,
    Err(e) => return Envelope::<()>::internal_error(Some(e.to_string())),
};

let items_storage = ItemsSqliteStorage::new(connection);
```

4. 3. Бизнес логика.

- **Рекомендуется выполнить** задание: «Envelope — обертка для возврата результата HTTP обработчика», перед выполнением этого шага.
- **Рекомендуется выполнить** задание: «Спецификация для гибких запросов», перед выполнением этого шага.
- **Рекомендуется выполнить** задание: «Конвертация типов вашего приложения в sql и наоборот», перед выполнением этого шага.

На этом шаге мы выполняем основную бизнес-логику запроса. В моем случае, это проверка наличия товара с таким же именем в базе данных, а потом сохранение нового товара в системе.

Сначала, мне нужно проверить, существует ли уже товар с таким же именем в базе данных.

```
let existing_item = match items_storage
    .get(ItemSpecification::new().with_name(name_clone))
    .await
{
    Err(e) => return Envelope::<()>::internal_error(Some(e.to_string())),
    Ok(item) => item,
};

if existing_item.is_some() {
    return Envelope::<()>::conflict(Some(
        AddItemError::DuplicateName.to_string()
    ))
}
```

После этого мне нужно сохранить новый товар в базе данных.

```
let item_id: ItemId = ItemId(EntityId::new());
let item_created_at: RecordLifetime = RecordLifetime::current_created();
let item = Item::restore(item_id, name, price, item_created_at);
match items_storage.add(item).await {
    Ok(_) => (),
    Err(e) => return Envelope::<()>::internal_error(Some(e.to_string())),
}
```

4. 4. Вернуть результат

Рекомендуется выполнить задание: «Envelope — обертка для возврата результата HTTP обработчика», перед выполнением этого шага.

В конце мне нужно вернуть результат:

```
Response::new(Status::OK).with_json(item_id.0.value())
```

Задачи

Задача 1. Определить бизнес логику приложения

Определите бизнес логику вашего приложения и создайте под них необходимые запросы, если они не существуют. Например:

- `add_item` — добавление нового товара в систему.
- `order_item` — заказ товара.
- `update_item` — обновление информации о товаре.
- `delete_item` — удаление товара из системы.

И так далее.

На каждую сущность у вас должен присутствовать функционал по взаимодействию с ним.

Задача 2. Улучшить обработку запросов

Используя пример и дополнительные материалы («спецификация для гибких запросов», «envelope для стандартизации единого формата ответа», «конвертация объектов-значений (value-objects) в sql-типы») улучшите ваш проект.

Критерии принятия этапа 9:

1. Реализованы HTTP обработчики, в которых есть:
   - Шаг валидации данных.
   - Работа с базой данных.
   - Бизнес логика.
2. Реализован `Envelope` для стандартизации ответов.
3. Реализованы конвертеры для преобразования объектов-значений в SQL-типы.
4. Реализован шаблон `спецификации` для гибких запросов на получение данных сущностей.